

Joint Optimization of Neural Radiance Fields and Continuous Camera Motion from a Monocular Video

Hoang Chuong Nguyen¹ Wei Mao Jose M. Alvarez² Miaomiao Liu¹
¹Australian National University ²NVIDIA
 {hoangchuong.nguyen, miaomiao.liu}@anu.edu.au josea@nvidia.com

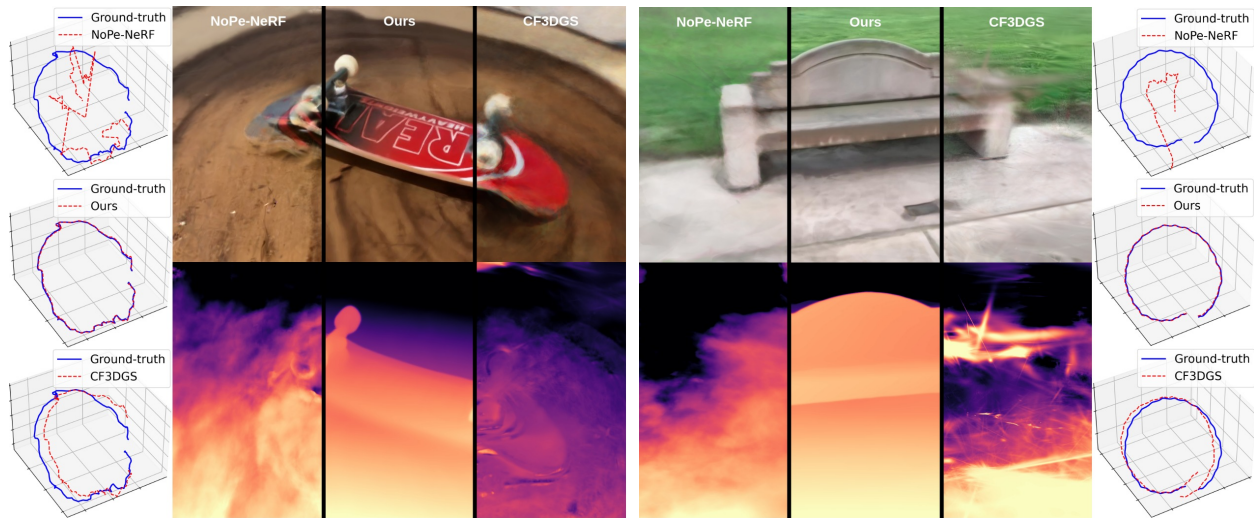


Figure 1. **Comparison with previous works** [2, 6]. Our method achieves superior performance in camera pose estimation (side), depth estimation (bottom) and novel-view synthesis (top).

Abstract

Neural Radiance Fields (NeRF) has demonstrated its superior capability to represent 3D geometry but require accurately precomputed camera poses during training. To mitigate this requirement, existing methods jointly optimize camera poses and NeRF often relying on good pose initialisation or depth priors. However, these approaches struggle in challenging scenarios, such as large rotations, as they map each camera to a world coordinate system. We propose a novel method that eliminates prior dependencies by modeling continuous camera motions as time-dependent angular velocity and velocity. Relative motions between cameras are learned first via velocity integration, while camera poses can be obtained by aggregating such relative motions up to a world coordinate system defined at a single time step within the video. Specifically, accurate continuous camera movements are learned through a time-dependent NeRF, which captures local scene geometry and motion by training from neighboring frames for each time

step. The learned motions enable fine-tuning the NeRF to represent the full scene geometry. Experiments on Co3D and Scannet show our approach achieves superior camera pose and depth estimation and comparable novel-view synthesis performance compared to state-of-the-art methods. Our code is available at <https://github.com/HoangChuongNguyen/cope-nerf>.

1. Introduction

Recent advances in Neural Radiance Fields (NeRF) [20, 29, 33] have demonstrated impressive performance in synthesizing photorealistic images and representing 3D scenes. The training process of these models requires known camera poses for training images, which are usually obtained using Structure-from-Motion methods such as COLMAP [26]. However, this process introduces additional computation, lacks differentiability, and is sensitive to hand-crafted features [1, 17].

To remove its reliance on COLMAP, existing meth-

ods have attempted to jointly optimize NeRF and camera poses [2, 16, 31]. Despite their promising results, they either are limited to forward-facing scenes [31] or rely on prior knowledge such as a good camera pose initialization [16], or monocular depth [2]. For the prior-based methods, their performance highly depends on the priors, and due to their inherent strategy of learning the frame-wise camera-to-world mappings, they also struggle to handle large camera motions.

In this paper, we propose a method to jointly optimize NeRF and camera poses which better leverages the temporal information of a monocular video *without relying on priors*. Specifically, instead of optimizing the discrete camera pose for each frame, we predict a continuous camera motion represented as angular velocity and velocity, capturing the temporal continuity and smoothness of the camera movements. Such continuous representation breaks camera motion down into infinitely small intervals, enabling large movements between any camera and world frame to be computed through the aggregation of motion over time via velocity integration. This approach significantly simplifies learning large camera motions, as shown in Fig. 1.

Given camera pose for each frame obtained through the velocity integration, one can train a single NeRF defined in a global world coordinate system, similar to existing approaches. However, camera poses are often noisy during the early training stage. Mapping 3D points from camera to the world coordinate system using these noisy poses makes the training harder and often stuck in local minimum. To address this, we propose to adopt a time-dependent NeRF to represent the scene at each time step, trained using neighboring frames around time t . While the time-dependent NeRF at each time step can only capture local scene geometry, it is sufficient for estimating continuous camera motion at the corresponding time step. Additionally, our modeling allows us to constrain the continuous camera motion and the time-dependent scene geometry represented by Signed Distance Field (SDF) with the linear relationship between the SDF flow and scene flow to improve the camera motion estimation as introduced in [18].

To model the full scene geometry, we fix the camera motion in the later training stage and continue training the time-dependent NeRF corresponding to the world coordinate system. Thanks to our continuous camera motion modeling, we can integrate the motion of each camera frame relative to the world coordinate system and train the NeRF using all available frames. This process enables us to reconstruct the entire scene geometry while improving rendering quality. Our proposed framework allows us to obtain an accurate camera pose for each frame as well as the 3D scene representation. In summary, our contributions are as follows:

- We introduce a prior-free pipeline to jointly optimize camera motion and scene geometry using NeRF.
- We propose to represent the camera motion in a continuous way, helping to learn large camera motions.
- We further propose to utilize a time-dependent NeRF, which allows the estimation of accurate camera motion within a small time window. It also enables the introduction of the linear relationship between the camera motion and scene geometry during training for further improving the camera motion estimation.

Our approach outperforms state-of-the-art methods in camera pose and depth estimation, and achieve comparable performance in novel-view synthesis.

2. Related Works

Scene reconstruction and rendering with known camera poses. Neural radiance fields (NeRF) [20], and 3D Gaussian splatting (3DGS) [12] have drawn attention in recent years due to their superiority and efficiency in rendering photo-realistic images and modeling 3D scenes. Extracting accurate 3D geometry with 3DGS remains an active research direction, with recent works introducing better rasterization strategies [35, 36] and additional regularizations [9, 10]. Regarding NeRF, several works have proposed to enhance neural implicit representations via SDF field [29, 33]. Additional efforts have been made for accelerating the rendering process [3, 21], reconstructing large scenes [27] or dynamic scenes [7, 18]. Despite their advancements, both these 3DGS-based and NeRF-based methods still rely on Structure-from-Motion (SfM) algorithms [26] to acquire the precomputed camera poses essential for training. Unlike these works, our proposed method can jointly optimize the camera poses and the scene geometry (represented via NeRF).

Joint 3DGS and poses optimization. Several works [6, 19, 25, 32] have explored the joint optimization of 3DGS and camera poses. These methods typically employ a SLAM-style pipeline to sequentially optimize relative camera transformation between consecutive frames in a video. Contrary to these methods, we differ by modeling the continuous camera motions instead of the relative camera motions. Such continuous motions enable us to enforce a constraint on the temporal changes of the observed scene geometry, thereby improving both the inferred camera poses and overall scene geometry. However, this cannot be achieved by learning only the relative camera mappings. Additionally, without geometric prior, 3DGS-based methods have demonstrated limited success in scene reconstruction [12, 19]. The key challenge of this stems from their Gaussians densification which can be challenging near regions with abrupt depth change. Moreover, many noisy Gaussians

can be grown when the poses are not fully optimized, potentially leading to memory issues during training [6, 13]. Additionally, the performance of 3DGS-based methods tends to degrade when accurate 3D points for initializing the Gaussians are not available [12, 19]. To overcome this, either ground-truth depth [11, 19, 32] or a pre-trained depth network [6, 25] is essential for initialization and guiding the densification process. In contrast, our method does not rely on additional geometric priors and still achieves accurate 3D reconstruction.

Joint NeRF and poses optimization. Differing from 3DGS, NeRF-based methods do not require accurate point clouds for initialization, yet still outperform 3DGS-based approaches in reconstructing the scene geometry [35]. Regarding the joint NeRF and poses optimization, a pioneering method in this direction is i-NeRF [34], which utilizes a pre-trained NeRF model to recover camera poses. NeRFmm [31] further advances this by jointly optimizing NeRF along with camera extrinsic and intrinsic parameters. BARF [16] and GARF [4] focus on addressing the negative impact of naive positional embeddings on pose estimation. While these methods have shown promising results, they are either limited to forward-facing scenes or require good pose initialization. NoPe-NeRF [2] tackles this by leveraging a pre-trained depth network [23]. However, it still struggles in dealing with large camera-to-world mappings. Unlike previous works, our method is prior-free and capable of handling large motions by exploiting temporal information from a monocular video.

In particular, we propose to model continuous camera motion and a time-dependent NeRF, which enables the learning of accurate short-term camera motions. As opposed to prior methods, our approach can avoid the direct optimization of large camera-to-world mappings, since the camera pose at any frame can be recovered by integrating the learned continuous camera motion up to a selected world frame. While the time-dependent NeRF is also leveraged in [7], it is used for motion predictions in a dynamic scene. On the other hand, in our pipeline, this model offers an efficient way to compute the SDF flows, which is important to constrain the learning of the continuous camera motion. Compared to [18] which integrates the SDF flows overtime to obtain SDF values, our approach in leveraging this information is more efficient, as we can avoid the time-consuming SDF flows integration by directly predicting the SDF and deriving the SDF flows through automatic differentiation [22].

3. Method

We tackle the problem of jointly optimizing camera motion and NeRF from a monocular video. Our goal is not limited to synthesizing photo-realistic images but also

to achieving accurate camera motion and scene geometry estimation. We assume no additional information beyond camera intrinsics and images from the input video.

Our method begins with the joint optimization of continuous camera motions (Sec. 3.2) and a time-dependent NeRF (Sec. 3.3). Fig. 2 depicts the overall pipeline of this joint optimization. Subsequently, camera poses are derived from the learned motions, which are used to fine-tune our previously trained model following the conventional NeRF training pipeline [29] (Sec. 3.4).

3.1. Preliminary: NeRF with SDF Presentation

In this work, we adopt the SDF-based NeRF model introduced in [29]. This method represents the scene appearance using a mapping function, $\mathbf{c} = \phi_c(\mathbf{x}, \mathbf{d})$, that takes a point $\mathbf{x} \in \mathbb{R}^3$ and a camera viewing direction $\mathbf{d} \in \mathbb{R}^2$ as inputs to predict color $\mathbf{c} \in \mathbb{R}^3$. Likewise, the scene geometry is modeled via $s = \phi_g(\mathbf{x})$ which maps a 3D point $\mathbf{x}$ to a signed distance value $s \in \mathbb{R}$. For each camera ray $\mathbf{r}(h) = \mathbf{o} + h\mathbf{d}$ with viewing direction $\mathbf{d}$ passing through a camera origin $\mathbf{o} \in \mathbb{R}^3$, a set of K 3D points can be sampled: $\{\mathbf{x}_i(\mathbf{r}) = \mathbf{r}(h_i) \mid i = 1, \dots, K, h_i < h_{i+1}\}$. The signed distance can be converted into density value σ using:

$$\sigma_i = \max\left(\frac{\Phi(s_i) - \Phi(s_{i+1})}{\Phi(s_i)}, 0\right), \quad (1)$$

where $\Phi(s) = \frac{1}{1+e^{-\gamma s}}$, and $\gamma \in \mathbb{R}$ is a learnable parameter and i denotes an index of a sampled point. The pixel's color can then be obtained via volume rendering:

$$\hat{\mathbf{C}}(\mathbf{r}) = \sum_{i=1}^K L_i \sigma_i \mathbf{c}_i, \quad (2)$$

with $L_i = \prod_{j=1}^{i-1} (1 - \sigma_j)$ being the accumulated transmittance along a ray.

3.2. Continuous Camera Motion

Previous methods [2, 16, 31] aim to jointly optimize independent camera poses and a NeRF model to explain the observed images $\{\mathbf{C}_t \in \mathbb{R}^{H \times W \times 3}\}_{t=1}^T$. However, these approaches struggle in scenarios with significant camera-to-world transformations even with monocular depth prior from a pre-trained model [2]. Instead of relying on any prior, we propose to model the continuous camera motion by leveraging the temporal information from a video sequence to estimate the camera motions within a small time interval, thereby avoiding the direct estimation of the camera-to-world transformations. Specifically, we model this continuous motions as angular velocities and velocities of the camera. An MLP ϕ_v is then used to predict these velocities as:

$$\boldsymbol{\omega}(t), \mathbf{v}(t) = \phi_v(t), \quad (3)$$

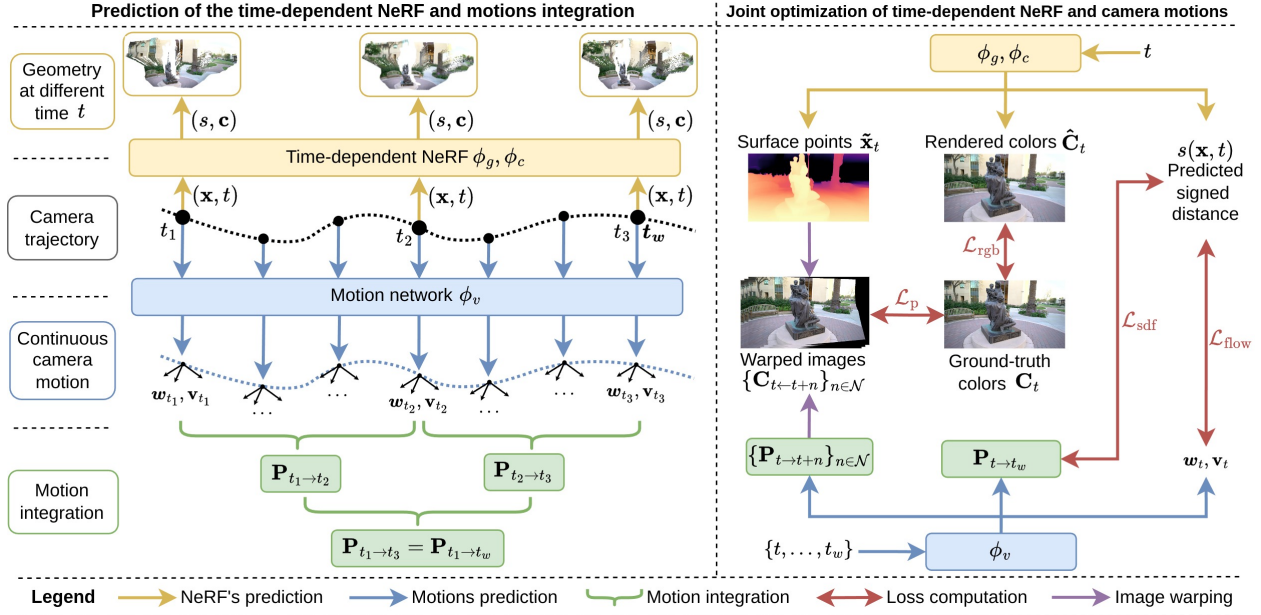


Figure 2. **Overview of our method.** **Left:** We jointly optimize the camera motion network ϕ_v to obtain the continuous camera motion represented as angular velocity ω and velocity $\mathbf{v}$, and a time-dependent NeRF (ϕ_g, ϕ_c) to obtain the scene geometry (represented as Signed Distance Field) and appearance at different time steps. The camera velocities can be integrated to obtain the camera transformation $\mathbf{P}$ between any two frames. Such relative camera motion is used to get 3D correspondences across different time steps. **Right:** The training our pipeline involves the standard rendering loss $\mathcal{L}_{\text{rgb}}$, and several consistency losses including, the consistency between SDF and camera motion $\mathcal{L}_{\text{flow}}$, the photometric consistency $\mathcal{L}_{\text{photo}}$, and the geometry consistency loss $\mathcal{L}_{\text{sdf}}$.

with $\omega(t), \mathbf{v}(t) \in \mathbb{R}^3$ being the angular velocity and the velocity of the camera at time t . The relationship between the camera transformation and such velocities can then be expressed as,

$$\frac{d\mathbf{R}}{dt} = [\omega(t)]_{\times} \mathbf{R}, \quad \frac{d\mathbf{t}}{dt} = \mathbf{v}(t), \quad (4)$$

where $\mathbf{R} \in SO(3)$ and $\mathbf{t} \in \mathbb{R}^3$ are the camera rotation and translation, respectively. $[\omega(t)]_{\times} \in so(3)$ is the skew-symmetric matrix of $\omega(t)$. In practice, we use Euler method to solve the above Ordinary Differential Equations. In particular, the relative rotation and translation from time t to $t+l$ ($l > 0$) can be computed as,

$$\mathbf{R}_{t \rightarrow t+l} = \prod_{u=0}^{U-1} \psi(\omega(t+u\Delta t)\Delta t) \quad (5)$$

$$\mathbf{t}_{t \rightarrow t+l} = \sum_{u=0}^{U-1} \mathbf{v}(t+u\Delta t)\Delta t, \quad (6)$$

where $\psi(\cdot) : \mathbb{R}^3 \rightarrow SO(3)$ converts rotation angles to rotation matrix and $\Delta t = \frac{l}{U}$ is the step size. Note that we apply right-matrix multiplication in Eq. 5. The transformation matrix that transform the coordinate of any point from any time step t_1 to another time step t_2 can

be defined as,

$$\mathbf{P}_{t_1 \rightarrow t_2} = \begin{cases} \mathbf{B}_{t_1 \rightarrow t_2} & \text{if } t_1 \leq t_2, \\ (\mathbf{B}_{t_2 \rightarrow t_1})^{-1} & \text{otherwise} \end{cases}, \quad (7)$$

where $\mathbf{B}_{t_1 \rightarrow t_2} = \begin{bmatrix} \mathbf{R}_{t_1 \rightarrow t_2} & \mathbf{t}_{t_1 \rightarrow t_2} \\ \mathbf{0} & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}$.

3.3. Time-dependent NeRF

Given the transformation matrices, an intuitive solution, as done in prior works [2, 16], is to train a single NeRF defined in the world coordinate system and map any 3D point to the world via these transformations. However, at early training stage, these transformations are very noisy which makes the optimization harder and often stuck in local minimum. To address this, we propose to relax the model from maintaining a global scene to many local ones by using a time-dependent NeRF to represent the scene at each time step. We then encourage the scene consistency across different time steps with soft constraints (i.e., losses). In particular, our NeRF model predicts the SDF $s(\mathbf{x}, t)$ and the color $\mathbf{c}(\mathbf{x}, t)$ for point $\mathbf{x}$ in the local camera coordinate system at time t as,

$$s(\mathbf{x}, t) = \phi_g(\mathbf{x}, t), \quad \mathbf{c}(\mathbf{x}, t) = \phi_c(\mathbf{x}, \mathbf{d}, t), \quad (8)$$

where $\mathbf{d} \in \mathbb{R}^2$ is a viewing direction in the camera coordinate system. We then perform volume rendering using

Eq. 2 to obtain the color $\hat{\mathbf{C}}_t(\mathbf{r}) \in \mathbb{R}^3$ for ray $\mathbf{r}$ at time t . The model is trained using the standard color loss and the Eikonal term [8] to constrain the SDF,

$$\mathcal{L}_{\text{rgb}} = \frac{1}{|\Omega|} \sum_{\mathbf{r} \in \Omega} \left\| \hat{\mathbf{C}}_t(\mathbf{r}) - \mathbf{C}_t(\mathbf{r}) \right\|_2, \quad (9)$$

$$\mathcal{L}_{\text{eik}} = \frac{1}{|\mathcal{X}|} \sum_{\mathbf{x} \in \mathcal{X}} \left(\left\| \frac{\partial s(\mathbf{x}, t)}{\partial \mathbf{x}} \right\|_2 - 1 \right)^2, \quad (10)$$

where Ω and $\mathcal{X}$ define a set of rays and sampled 3D points, respectively. In addition, we propose to use a series of losses, including $\mathcal{L}_{\text{flow}}$, $\mathcal{L}_{\text{photo}}$, $\mathcal{L}_{\text{sdf}}$ (see definition below), to encourage the consistency between the scene geometry across time, and the camera motion. Since the time-dependent NeRF models local scene at each time step, without constraint, there are inconsistencies in geometry for overlapping regions. To avoid this, we first leverage the linear relationship between scene flow and SDF flow derived in [18] to constrain the camera motion and the scene geometry as,

$$\mathcal{L}_{\text{flow}} = \frac{1}{|\mathcal{S}|} \sum_{\mathbf{x} \in \mathcal{S}} \left| \frac{\partial s(\mathbf{x}, t)}{\partial t} + (\boldsymbol{\omega}(t) \times \mathbf{x} + \mathbf{v}(t))^T \mathbf{n}(\mathbf{x}, t) \right|, \quad (11)$$

where $\mathbf{n}(\mathbf{x}) = \frac{\partial s(\mathbf{x}, t)}{\partial \mathbf{x}} \in \mathbb{R}^3$ is the surface normal at $\mathbf{x}$ and $\mathcal{S}$ is the set of surface points. Intuitively, this loss constrains the scene change at different surface points to be consistent with the same camera motion at any time step, thus encouraging the scene surface to change rigidly with respect to time. For more details, please refer to the original paper [18].

We also utilize the photometric consistency loss from the literature on monocular depth estimation to supervise the camera motion and the time-dependent NeRF. To this end, we first obtain the surface point $\tilde{\mathbf{x}}(\mathbf{r}, t) \in \mathbb{R}^3$ at time step t of ray $\mathbf{r}$ using volume rendering as,

$$\tilde{\mathbf{x}}(\mathbf{r}, t) = \sum_{i=1}^K L_i \sigma_i \mathbf{x}_i(\mathbf{r}, t). \quad (12)$$

We can then project the surface point to neighboring frames using the estimated camera pose,

$$\tilde{\mathbf{x}}'_{t \rightarrow t+n}(\mathbf{r}, t) = \mathbf{P}_{t \rightarrow t+n} \tilde{\mathbf{x}}'(\mathbf{r}, t), \quad (13)$$

where $n \in \mathcal{N}$ and $\mathcal{N}$ is a hyper-parameter defining a set of intervals to neighboring frames. Here $\tilde{\mathbf{x}}'_{t \rightarrow t+n}(\mathbf{r}, t)$ and $\tilde{\mathbf{x}}'(\mathbf{r}, t)$ denote the homogeneous coordinate of the corresponding points, $\tilde{\mathbf{x}}_{t \rightarrow t+n}(\mathbf{r}, t)$ and $\tilde{\mathbf{x}}(\mathbf{r}, t)$, respectively. Given these corresponding points, the photometric consistency loss is computed as,

$$\mathcal{L}_{\text{photo}} = \frac{1}{|\mathcal{N}||\Omega|} \sum_{n, \mathbf{r}} \left\| \mathbf{C}_{t+n}[\mathbf{K} \tilde{\mathbf{x}}_{t \rightarrow t+n}] - \mathbf{C}_t(\mathbf{r}) \right\|_1, \quad (14)$$

where $\mathbf{K}$ defines the intrinsic matrix and $[\cdot]$ denotes the sampling operation.

Lastly, we impose an SDF consistency loss $\mathcal{L}_{\text{sdf}}$ to encourage the consistency between the SDF values predicted in any frame and those in the world frame. In practice, we choose the middle frame as the world frame.

$$\mathcal{L}_{\text{sdf}} = \frac{1}{|\mathcal{X}|} \sum_{\mathbf{x} \in \mathcal{X}} |s(\mathbf{x}, t) - s(\mathbf{x}_{t \rightarrow t_w}, t_w)|, \quad (15)$$

where t_w is the time step corresponding to the world frame, and $\mathbf{x}_{t \rightarrow t_w}$ is the corresponding point of $\mathbf{x}$ at the world frame computed via Eq. 13.

3.4. Training

The overall loss used to train our model is,

$$\mathcal{L} = \mathcal{L}_{\text{rgb}} + \lambda_1 \mathcal{L}_{\text{eik}} + \lambda_2 \mathcal{L}_{\text{flow}} + \lambda_3 \mathcal{L}_{\text{photo}} + \lambda_4 \mathcal{L}_{\text{sdf}}, \quad (16)$$

where $\{\lambda_i\}_{i=1}^4$ are weights. Note that since the inferred camera-to-world transformations are initially noisy, we set the SDF consistency loss weight (λ_4) to be 0 for the first 200 epochs, and gradually increase it after that.

At the early stage of our training, we jointly train all models (ϕ_v, ϕ_g, ϕ_c) to first obtain accurate camera motions, and the scene geometry encoded in different time steps. For the later stage of our training, we fix t to the world time step t_w and proceed with the conventional NeRF training pipeline [20] while keeping the camera poses $\mathbf{P}_{t \rightarrow t_w}$ fixed and only updating the scene geometry and appearance network (ϕ_g, ϕ_c) using all training frames in the video sequence. At this stage, we drop the loss weights $\lambda_2, \lambda_3, \lambda_4$ to 0.

After the training, the full scene geometry and appearance are obtained at the world frame. It is also worth noting that our training pipeline is end-to-end trainable and does not rely on any prior knowledge about the camera motion or the scene.

4. Experiments

4.1. Experimental Setup

Dataset. Following [2, 6], we evaluate our method using 4 scenes from ScanNet [5] dataset and 5 scenes from Co3D [24] dataset. Each scene corresponds to a video containing a varying number of images. Similar to prior works, every 8th image in each video is selected for novel-view synthesis and depth evaluation, whereas the rest are used for training. For both datasets, we utilize the provided poses and depth maps as ground-truth. While Scannet is captured in indoor environment, Co3D features a combination of indoor and outdoor scenes with more complex camera motions (i.e., large rotations). Moreover, we include results for the Tanks and Temples [15] dataset in the supplementary materials.

	Scenes	Ours			NeRFmm ^{††} [31]			NoPe-NeRF ^{*†} (D) [2]			CF3DGS ^{††} (D) [6]			CF3DGS ^{**} (D) [6]		
		AbRel ↓	SqRel ↓	δ_1 ↑	AbRel	SqRel	δ_1	AbRel	SqRel	δ_1	AbRel	SqRel	δ_1	AbRel	SqRel	δ_1
Scannet	0079	0.033	0.004	0.995	0.184	0.113	0.682	0.099	0.047	0.904	<u>0.078</u>	<u>0.018</u>	<u>0.973</u>	–	–	–
	0418	0.144	0.067	0.853	0.611	1.046	0.195	<u>0.152</u>	<u>0.137</u>	<u>0.738</u>	0.239	0.164	0.593	–	–	–
	0301	0.037	0.012	0.977	0.223	0.208	0.562	<u>0.185</u>	0.252	<u>0.792</u>	0.201	<u>0.229</u>	0.745	–	–	–
	0431	0.038	0.016	0.982	0.258	0.187	0.503	0.127	0.111	0.877	<u>0.108</u>	<u>0.052</u>	<u>0.900</u>	–	–	–
Co3D	Bench	0.023	0.085	0.981	0.291	1.866	0.467	<u>0.183</u>	<u>0.945</u>	0.692	0.208	3.028	<u>0.738</u>	–	–	–
	Skateboard	0.012	0.013	0.998	0.179	0.539	0.674	0.159	<u>0.527</u>	0.768	<u>0.138</u>	1.134	<u>0.865</u>	–	–	–
	Plant	0.074	0.526	0.933	0.337	4.145	0.444	0.249	<u>1.816</u>	0.538	0.201	1.831	<u>0.650</u>	–	–	–
	Hydrant	0.022	0.064	0.988	0.402	4.281	0.260	<u>0.154</u>	<u>0.677</u>	<u>0.764</u>	0.206	1.950	0.651	–	–	–
	Teddy	0.026	0.175	0.977	0.254	1.790	0.474	<u>0.135</u>	<u>0.823</u>	<u>0.842</u>	0.302	10.08	0.754	–	–	–

Table 1. **Depth evaluation.** Our method achieves the best results across both datasets. (D) indicates methods leveraging depth priors. For each method, we report its released results if available (*). Otherwise, we train it using published implementation (†). For example, NoPe-NeRF^{*†} means we report its released results on Scannet and train it using author’s code for the Co3D dataset.

	Scenes	Ours			NeRFmm ^{††} [31]			NoPe-NeRF ^{*†} (D) [2]			CF3DGS ^{††} (D) [6]			CF3DGS ^{**} (D) [6]		
		PSNR ↑	SSIM ↑	LPIPS ↓	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Scannet	0079	35.52	0.92	<u>0.20</u>	32.06	<u>0.85</u>	0.33	32.47	0.84	0.41	33.91	0.92	0.16	–	–	–
	0418	34.53	0.92	<u>0.29</u>	30.92	<u>0.78</u>	0.30	31.33	0.79	0.34	32.56	0.90	0.20	–	–	–
	0301	32.06	0.88	<u>0.22</u>	30.10	<u>0.83</u>	0.29	29.83	0.77	0.36	31.09	0.88	0.20	–	–	–
	0431	34.25	0.93	<u>0.25</u>	32.94	0.90	0.31	<u>33.83</u>	<u>0.91</u>	0.39	31.02	0.90	0.22	–	–	–
Co3D	Bench	26.36	0.68	0.40	23.31	0.58	0.55	24.32	0.60	0.52	25.06	<u>0.70</u>	<u>0.33</u>	26.21	0.73	0.32
	Skateboard	30.93	0.87	<u>0.32</u>	23.10	0.75	0.47	26.22	0.80	0.42	22.21	0.76	0.36	27.24	0.85	0.30
	Plant	<u>26.53</u>	<u>0.77</u>	<u>0.41</u>	22.26	0.68	0.52	23.79	0.71	0.48	20.73	0.67	0.42	29.69	0.89	0.29
	Hydrant	<u>20.60</u>	<u>0.51</u>	0.44	19.11	0.41	0.59	19.82	0.42	0.57	19.22	0.50	<u>0.40</u>	22.14	0.64	0.34
	Teddy	33.04	0.89	0.20	27.87	0.77	0.42	<u>29.40</u>	0.80	0.38	24.13	0.77	0.27	27.75	<u>0.86</u>	0.20

Table 2. **Image evaluation.** Our approach significantly outperforms prior NeRF-based methods and is on par with CF3DGS. (D) indicates methods leveraging depth priors. For each method, we report its released results if available (*). Otherwise, we train it using published implementation (†).

Metrics. We follow a similar evaluation scheme as [2]. Specifically, the metrics used for novel-view synthesis are PSNR, SSIM [30] and LPIPS [37]. Regarding pose evaluation, we use RPE_t , RPE_r and ATE to measure the errors of the relative translations, relative rotations, and camera trajectories, respectively. Prior to computing these metrics, we apply Umeyama alignment [28] to align the estimated and ground-truth poses. For depth evaluation, we report three metrics including the absolute relative error (AbRel), square relative error (SqRel) and a depth accuracy metric (δ_1). Since the rendered depths are up-to-scale, we follow [38] to recover the ground-truth scale before calculating the depth metrics.

Implementation Details. Our NeRF architecture is based on [29] with two modifications: (1) We use the SDF-network to represent the entire scene instead of employing an extra model [20] for the background, and (2) our model takes a timestep as an additional input. The motion network has a similar structure to the SDF-network, with the exception of the input and output layers. The input timesteps for all models are normalized from -1 to 1 . We use Adam [14] optimizer with a learning rate of 0.001 to train the time-dependent NeRF and

the motion network until convergence. Then, we fine-tune our NeRF model following the conventional training pipeline [20] for an additional 5000 epochs with the learning rate decaying gradually. At each training iteration, we sample 1024 rays and 128 3D points per ray within a pre-defined depth range. For all scenes, we use 10 sub-intervals between two consecutive frames to solve for the relative camera transformation between them (Eq. 5 and 6), and the world time step t_w is chosen as the middle frame in the training set. More details regarding the datasets, evaluation metrics and other hyperparameters can be found in the supplementary materials.

4.2. Results

We compare our method with two other NeRF-based methods: NeRFmm [31] and NoPe-NeRF [2], and a recent 3DGS-based method: CF3DGS [6]. Among these, our method and NeRFmm take only RGB images from a monocular video as input, whereas NoPe-NeRF and CF3DGS leverage additional geometric information from a pre-trained depth network.

Geometry. We quantify the geometric accuracy of each method through depth evaluation in Tab. 1. Our

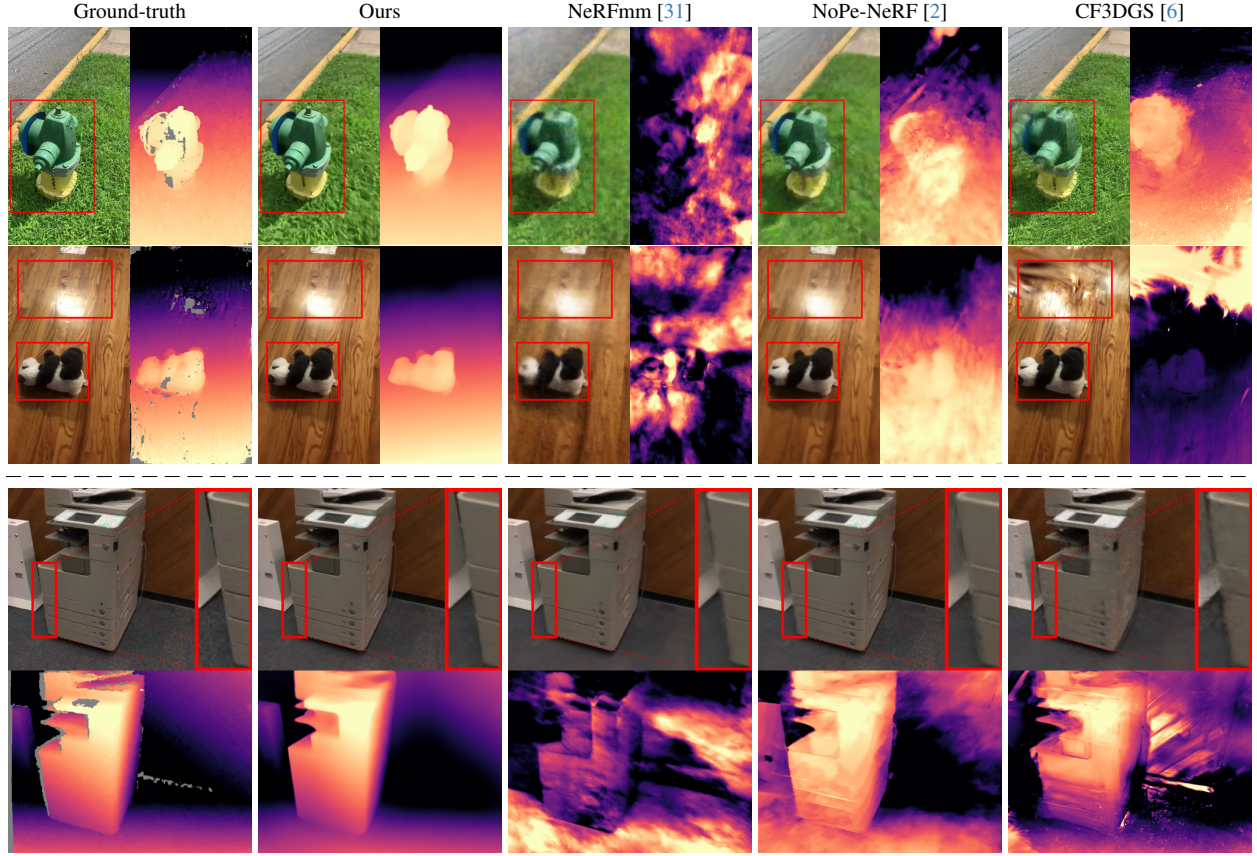


Figure 3. **Qualitative results on the Co3D (top) and Scannet (bottom) dataset.** Our synthesized images are more photo-realistic compared to the other methods. In terms of geometry, our method produces the most accurate depth maps among all methods.

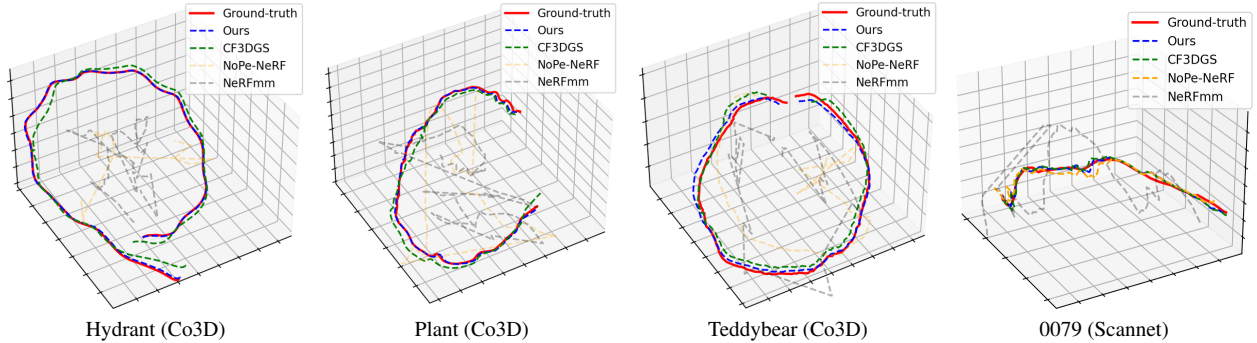


Figure 4. **Camera trajectory visualization.** Our poses are better aligned with the ground-truth compared to the others.

method consistently outperforms other methods with notable margins across all scenes in both datasets. On average, our method reduces the depth square relative errors by at least **82%** and **78%** on the Co3D and Scannet datasets, respectively. Fig. 3 reveals that although both NoPe-NeRF and CF3DGS leverage depth priors, they still fail to render accurate depth maps. On the other hand, our method can produce correct scene geometry given only RGB images during training.

Novel-view Synthesis. To obtain the camera poses corresponding to the test images, we keep the NeRF model fixed and learn these poses by minimizing the color rendering loss $\mathcal{L}_{\text{rgb}}$, as in [2]. The results in Tab. 2 show that our method significantly outperforms prior NeRF-based methods and is comparable to CF3DGS. However, as can be seen in Fig. 1 and Fig. 3, CF3DGS produces clear artifacts in all of its rendered images on the Co3D dataset. On the other hand, our synthesized images are

	Scenes	Ours			NeRFmm ^{††} [31]			NoPe-NeRF* [†] (D) [2]			CF3DGS ^{††} (D) [6]			CF3DGS** (D) [6]		
		RPE _t ↓	RPE _r ↓	ATE ↓	RPE _t	RPE _r	ATE	RPE _t	RPE _r	ATE	RPE _t	RPE _r	ATE	RPE _t	RPE _r	ATE
Scannet	0079	0.664	0.182	0.013	3.312	0.801	0.151	0.752	0.204	0.023	<u>0.673</u>	<u>0.190</u>	<u>0.015</u>	–	–	–
	0418	0.401	0.119	0.015	1.428	0.396	0.016	<u>0.455</u>	0.119	0.015	0.529	0.131	0.019	–	–	–
	0301	0.367	0.117	0.009	1.852	0.591	0.296	0.399	0.123	<u>0.013</u>	<u>0.393</u>	<u>0.118</u>	0.015	–	–	–
	0431	1.097	0.223	0.042	6.661	0.574	0.475	1.625	0.274	0.069	<u>1.301</u>	<u>0.269</u>	<u>0.064</u>	–	–	–
Co3D	Bench	0.013	0.037	0.001	0.562	3.508	0.039	0.301	1.925	0.053	<u>0.059</u>	<u>0.357</u>	0.017	0.110	0.424	0.014
	Skateboard	0.024	0.063	0.001	0.702	2.794	0.057	0.421	1.883	0.048	<u>0.063</u>	<u>0.462</u>	<u>0.017</u>	0.239	0.472	0.017
	Plant	0.014	0.062	0.002	0.650	2.557	0.059	0.305	1.587	0.047	<u>0.049</u>	<u>0.465</u>	<u>0.006</u>	0.140	<u>0.401</u>	0.021
	Hydrant	0.010	0.027	0.001	0.497	2.556	0.063	0.337	1.557	0.060	<u>0.059</u>	<u>0.344</u>	<u>0.008</u>	0.094	0.360	0.008
	Teddy	0.053	0.129	0.004	0.588	3.025	0.049	0.286	1.295	0.040	<u>0.072</u>	<u>0.184</u>	<u>0.009</u>	0.505	0.211	<u>0.009</u>

Table 3. **Pose evaluation.** Our method achieves the lowest pose errors in all scenes. (D) indicates methods leveraging depth priors. For each method, we report its released results if available (*). Otherwise, we train it using published implementation (†).

	Depth		Pose			Image	
	AbRel ↓	δ_1 ↑	RPE _t ↓	RPE _r ↓	ATE ↓	PSNR ↑	SSIM ↑
Full	0.031	0.975	0.023	0.063	0.002	<u>27.49</u>	<u>0.74</u>
w/o $\mathcal{L}_{\text{flow}}$	0.084	0.877	0.114	0.371	0.008	26.31	0.72
w/o $\mathcal{L}_{\text{sdf}}$	<u>0.072</u>	<u>0.913</u>	0.031	<u>0.094</u>	<u>0.004</u>	26.54	0.72
w/o $\mathcal{L}_{\text{photo}}$	0.399	0.386	0.216	1.402	0.041	21.79	0.65
w/o NeRF _t	0.287	0.504	0.299	1.808	0.045	24.08	0.67
w/o NeRF _{t→c}	0.082	0.857	–	–	–	28.29	0.76
w/o motion net	0.080	0.912	<u>0.028</u>	0.154	0.006	26.23	0.71

Table 4. Ablation study on the Co3D dataset.

visually more photorealistic than those of CF3DGS.

Camera Poses. Tab. 3 highlights the superior performance of our method in recovering the camera poses. While there are minor improvements on Scannet dataset, our method demonstrates a clear dominance in all challenging scenes in the Co3D dataset. Specifically, on Co3D, we achieve a minimum of **83%**, **63%** and **83%** error reductions in estimating the camera trajectory, relative camera translation and relative camera rotation, respectively. The inferior performance of NeRFmm and NoPe-NeRF is primarily due to their direct optimization of the camera-to-world transformations, which is challenging in case of large camera motions. In contrast, our method can avoid this issue and correctly recover the camera poses by modeling continuous camera motions. It can be observed from Fig. 4 that our learned camera trajectories are better aligned with the ground truth compared to the other methods. Additionally, we also provide a comparison with COLMAP in the supplementary materials, demonstrating that our method yields more accurate camera poses.

4.3. Ablation Study

In this section, we conduct an ablation study to demonstrate the effectiveness of each component in our proposed method. The results are shown in Tab. 4.

w/o $\mathcal{L}_{\text{flow}}$: Without $\mathcal{L}_{\text{flow}}$, there is a significant performance drop in all aspects. **w/o $\mathcal{L}_{\text{sdf}}$:** Training without the SDF consistency loss $\mathcal{L}_{\text{sdf}}$ leads to performance

degradation. **w/o $\mathcal{L}_{\text{photo}}$:** Dropping the photometric loss results in the largest errors in all metrics. This illustrates the effectiveness of the losses used in our method.

w/o NeRF_t: Instead of leveraging a time-dependent NeRF, we aggregate the predicted motions to obtain the camera poses, and use them to train a conventional NeRF [29]. As the aggregated motions are initially noisy, this strategy results in the large errors across all metrics. **w/o NeRF_{t→c}:** Instead of fine-tuning the time-dependent NeRF in the later training stage, we use our learned camera poses to train a new conventional NeRF model from scratch. This approach slightly improves the quality of the rendered images but causes an increase of roughly 160% in the depth error. **w/o motion net:** Here, we remove the motion network and optimize the camera poses as learnable variables. Consequently, the error increases significantly. These results highlight the significance of the time-dependent NeRF and the continuous motion modeling in our pipeline.

5. Conclusion

In this work, we present a prior-free pipeline for the joint optimization of camera poses and NeRF from a monocular video. The key contributions are to model continuous camera motions and adopt a time-dependent NeRF to avoid the challenge of directly optimizing large camera-to-world mappings and obtain accurate geometry. Extensive experiments on Co3D and Scannet dataset demonstrate that our method outperforms previous methods in terms of pose estimation and depth estimation, while achieving comparable performance in novel-view synthesis.

Limitation. Since our method relies on visual cues in the training images to learn the camera motions, the joint optimization can become challenging in scenes having large low-texture regions or reflective surfaces.

Acknowledgement. This research was supported in part by the Australia Research Council ARC Discovery Grant (DP200102274).

References

- [1] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. Surf: Speeded up robust features. In *Computer Vision—ECCV 2006: 9th European Conference on Computer Vision, Graz, Austria, May 7–13, 2006. Proceedings, Part I* 9, pages 404–417. Springer, 2006. 1
- [2] Wenjing Bian, Zirui Wang, Kejie Li, Jia-Wang Bian, and Victor Adrian Prisacariu. Nope-nerf: Optimising neural radiance field with no pose prior. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4160–4169, 2023. 1, 2, 3, 4, 5, 6, 7, 8
- [3] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. Tensorf: Tensorial radiance fields. In *European conference on computer vision*, pages 333–350. Springer, 2022. 2
- [4] Shin-Fang Chng, Sameera Ramasinghe, Jamie Sherrah, and Simon Lucey. Gaussian activated neural radiance fields for high fidelity reconstruction and pose estimation. In *European Conference on Computer Vision*, pages 264–280. Springer, 2022. 3
- [5] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5828–5839, 2017. 5
- [6] Yang Fu, Sifei Liu, Amey Kulkarni, Jan Kautz, Alexei A. Efros, and Xiaolong Wang. Colmap-free 3d gaussian splatting. 2023. 1, 2, 3, 5, 6, 7, 8
- [7] Chen Gao, Ayush Saraf, Johannes Kopf, and Jia-Bin Huang. Dynamic view synthesis from dynamic monocular video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5712–5721, 2021. 2, 3
- [8] Amos Gropp, Lior Yariv, Niv Haim, Matan Atzmon, and Yaron Lipman. Implicit geometric regularization for learning shapes. *arXiv preprint arXiv:2002.10099*, 2020. 5
- [9] Antoine Guédon and Vincent Lepetit. Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5354–5363, 2024. 2
- [10] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In *ACM SIGGRAPH 2024 Conference Papers*, pages 1–11, 2024. 2
- [11] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. Splatam: Splat track & map 3d gaussians for dense rgb-d slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21357–21366, 2024. 3
- [12] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. 2, 3
- [13] Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Jeff Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *arXiv preprint arXiv:2404.09591*, 2024. 3
- [14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. 6
- [15] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics (ToG)*, 36(4):1–13, 2017. 5
- [16] Chen-Hsuan Lin, Wei-Chiu Ma, Antonio Torralba, and Simon Lucey. Barf: Bundle-adjusting neural radiance fields. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5741–5751, 2021. 2, 3, 4
- [17] David G Lowe. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60:91–110, 2004. 1
- [18] Wei Mao, Richard Hartley, Mathieu Salzmann, and Miaomiao Liu. Neural sdf flow for 3d reconstruction of dynamic scenes. In *The Twelfth International Conference on Learning Representations*. 2, 3, 5
- [19] Hidenobu Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. Gaussian Splatting SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 2, 3
- [20] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 1, 2, 5, 6
- [21] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)*, 41(4):1–15, 2022. 2
- [22] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017. 3
- [23] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 12179–12188, 2021. 3
- [24] Jeremy Reizenstein, Roman Shapovalov, Philipp Henzler, Luca Sbordone, Patrick Labatut, and David Novotny. Common objects in 3d: Large-scale learning and evaluation of real-life 3d category reconstruction. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10901–10911, 2021. 5
- [25] Erik Sandström, Keisuke Tateno, Michael Oechsle, Michael Niemeyer, Luc Van Gool, Martin R Oswald, and Federico Tombari. Splat-slam: Globally optimized rgb-only slam with 3d gaussians. *arXiv preprint arXiv:2405.16544*, 2024. 2, 3
- [26] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4104–4113, 2016. 1, 2

- [27] Matthew Tancik, Vincent Casser, Xincheng Yan, Sabeek Pradhan, Ben Mildenhall, Pratul P Srinivasan, Jonathan T Barron, and Henrik Kretzschmar. Block-nerf: Scalable large scene neural view synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8248–8258, 2022. [2](#)
- [28] Shinji Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 13(04):376–380, 1991. [6](#)
- [29] Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. Neus: Learning neural implicit surfaces by volume rendering for multi-view reconstruction. *arXiv preprint arXiv:2106.10689*, 2021. [1](#), [2](#), [3](#), [6](#), [8](#)
- [30] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004. [6](#)
- [31] Zirui Wang, Shangzhe Wu, Weidi Xie, Min Chen, and Victor Adrian Prisacariu. Nerf-: Neural radiance fields without known camera parameters. *arXiv preprint arXiv:2102.07064*, 2021. [2](#), [3](#), [6](#), [7](#), [8](#)
- [32] Chi Yan, Delin Qu, Dan Xu, Bin Zhao, Zhigang Wang, Dong Wang, and Xuelong Li. Gs-slam: Dense visual slam with 3d gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19595–19604, 2024. [2](#), [3](#)
- [33] Lior Yariv, Jiatao Gu, Yoni Kasten, and Yaron Lipman. Volume rendering of neural implicit surfaces. *Advances in Neural Information Processing Systems*, 34: 4805–4815, 2021. [1](#), [2](#)
- [34] Lin Yen-Chen, Pete Florence, Jonathan T Barron, Alberto Rodriguez, Phillip Isola, and Tsung-Yi Lin. inerf: Inverting neural radiance fields for pose estimation. in 2021 ieee. In *RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1323–1330. [3](#)
- [35] Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient and compact surface reconstruction in unbounded scenes. *arXiv preprint arXiv:2404.10772*, 2024. [2](#), [3](#)
- [36] Baowen Zhang, Chuan Fang, Rakesh Shrestha, Yixun Liang, Xiaoxiao Long, and Ping Tan. Rade-gs: Rasterizing depth in gaussian splatting. *arXiv preprint arXiv:2406.01467*, 2024. [2](#)
- [37] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. [6](#)
- [38] Tinghui Zhou, Matthew Brown, Noah Snavely, and David G Lowe. Unsupervised learning of depth and ego-motion from video. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1851–1858, 2017. [6](#)